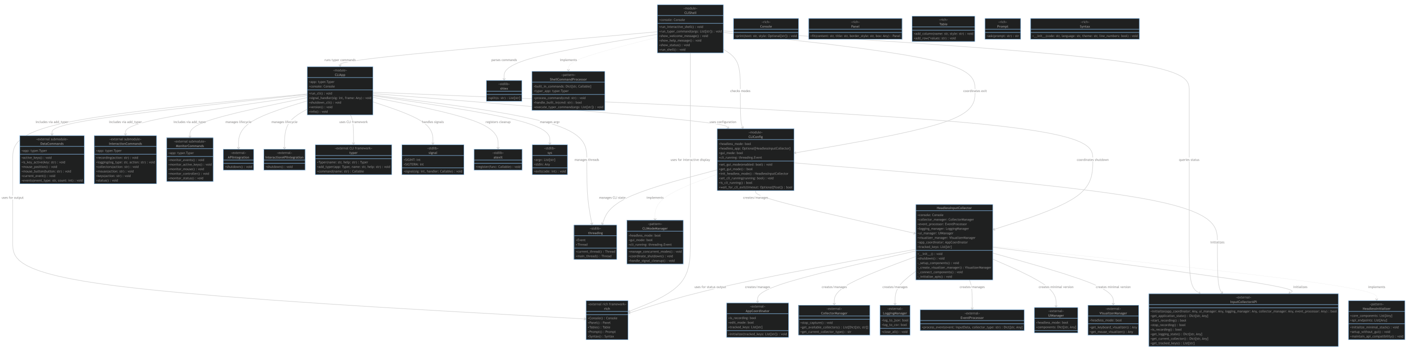
«module»
CLIShell
+console: Console
+run_interactive_shell() : void
+run_typer_command(args: List[str]) : void
+show_welcome_message() : void
+show_help_message() : void
+show_status() : void
+run_shell() : void
«rich»
Console
+print(text: str, style: Optional[str]) : void
«rich»
Panel
+fit(content: str, title: str, border_style: str, box: Any) : Panel
«rich»
Table
+add_column(name: str, style: str) : void
+add_row(*values: str) : void
«rich»
Prompt
+ask(prompt: str) : str
«rich»
Syntax
+__init__(code: str, language: str, theme: str, line_numbers: bool) : void
runs typer commands
parses commands
implements
checks modes
coordinates exit
«module»
CLIApp
+app: typer.Typer
+console: Console
+run_cli() : void
+signal_handler(sig: int, frame: Any) : void
+shutdown_cli() : void
+version() : void
+info() : void
«stdlib»
shlex
+split(s: str) : List[str]
«pattern»
ShellCommandProcessor
+built_in_commands: Dict[str, Callable]
+typer_app: typer.Typer
+process_command(cmd: str) : void
+handle_built_in(cmd: str) : bool
+execute_typer_command(args: List[str]) : void
includes via add_typer
includes via add_typer
includes via add_typer
manages lifecycle
manages lifecycle
uses CLI framework
handles signals
registers cleanup
manages argv
uses configuration
«external submodule»
DataCommands
+app: typer.Typer
+active_keys() : void
+is_key_active(key: str) : void
+mouse_position() : void
+mouse_button(button: str) : void
+current_event() : void
+events(event_type: str, count: int) : void
«external submodule»
InteractionCommands
+app: typer.Typer
+recording(action: str) : void
+logging(log_type: str, action: str) : void
+collectors(action: str) : void
+mouse(action: str) : void
+keys(action: str) : void
+status() : void
«external submodule»
MonitorCommands
+app: typer.Typer
+monitor_events() : void
+monitor_active_keys() : void
+monitor_mouse() : void
+monitor_controller() : void
+monitor_status() : void
«external»
APIIntegration
+shutdown() : void
«external»
InteractionAPIIntegration
+shutdown() : void
«external CLI framework»
typer
+Typer(name: str, help: str) : Typer
+add_typer(app: Typer, name: str, help: str) : void
+command(name: str) : Callable
«stdlib»
signal
+SIGINT: int
+SIGTERM: int
+signal(sig: int, handler: Callable) : void
«stdlib»
atexit
+register(func: Callable) : void
«stdlib»
sys
+argv: List[str]
+stdin: Any
+exit(code: int) : void
«module»
CLIConfig
+headless_mode: bool
+headless_app: Optional[HeadlessInputCollector]
+gui_mode: bool
+cli_running: threading.Event
+set_gui_mode(enabled: bool) : void
+get_gui_mode() : bool
+init_headless_mode() : HeadlessInputCollector
+set_cli_running(running: bool) : void
+is_cli_running() : bool
+wait_for_cli_exit(timeout: Optional[float]) : bool
uses for output
manages threads
uses for interactive display
coordinates shutdown
queries status
manages CLI state
implements
creates/manages
initializes
«stdlib»
threading
+Event
+Thread
+current_thread() : Thread
+main_thread() : Thread
«pattern»
CLIModeManager
+headless_mode: bool
+gui_mode: bool
+cli_running: threading.Event
+manage_concurrent_modes() : void
+coordinate_shutdown() : void
+handle_signal_cleanup() : void
HeadlessInputCollector
+console: Console
+collector_manager: CollectorManager
+event_processor: EventProcessor
+logging_manager: LoggingManager
+ui_manager: UIManager
+visualizer_manager: VisualizerManager
+app_coordinator: AppCoordinator
+tracked_keys: List[str]
+__init__() : void
+shutdown() : void
-_setup_components() : void
-_create_visualizer_manager() : VisualizerManager
-_connect_components() : void
-_initialize_apis() : void
uses for status output
creates/manages
creates/manages
creates/manages
creates/manages
creates minimal version
creates minimal version
initializes
implements
«external rich framework»
rich
+Console() : Console
+Panel() : Panel
+Table() : Table
+Prompt() : Prompt
+Syntax() : Syntax
«external»
AppCoordinator
+is_recording: bool
+edit_mode: bool
+tracked_keys: List[str]
+initialize(tracked_keys: List[str]) : void
«external»
CollectorManager
+stop_capture() : void
+get_available_collectors() : List[Dict[str, str]]
+get_current_collector_type() : str
«external»
LoggingManager
+log_to_json: bool
+log_to_csv: bool
+close_all() : void
«external»
EventProcessor
+process_event(event: InputData, collector_type: str) : Dict[str, Any]
«external»
UIManager
+headless_mode: bool
+components: Dict[str, Any]
«external»
VisualizerManager
+headless_mode: bool
+get_keyboard_visualizer() : Any
+get_mouse_visualizer() : Any
«external»
InputCollectorAPI
+initialize(app_coordinator: Any, ui_manager: Any, logging_manager: Any, collector_manager: Any, event_processor: Any) : bool
+get_application_state() : Dict[str, Any]
+start_recording() : bool
+stop_recording() : bool
+is_recording() : bool
+get_logging_state() : Dict[str, Any]
+get_current_collector() : Dict[str, Any]
+get_tracked_keys() : List[str]
«pattern»
HeadlessInitializer
+core_components: List[Any]
+api_endpoints: List[Any]
+initialize_minimal_stack() : void
+setup_without_gui() : void
+maintain_api_compatibility() : void